Auditoría técnica de ceramicconnection.com

Fecha: 14/10/2026 · Rama analizada: `devel` · PrestaShop 1.7.8.11 con tema `child_classic`

El documento tiene dos partes:

1. **Informe**: qué se ha revisado, cómo, y qué se ha encontrado, con el fichero y la línea de cada hallazgo.
2. **Tareas**: lista de tareas pequeñas, listas para copiar a Jira o ClickUp. Cada una explica qué hay que hacer, dónde, por qué, cómo y cuándo se da por terminada. Hay un CSV con las mismas tareas para importarlas directamente (`auditoria-web-2026-10-14-tareas.csv`).

Parte 1 · Informe

1.1 Alcance y método

Qué se ha revisado

- Los overrides (`/override`).
- El tema hijo (`themes/child_classic` : plantillas, `custom.js` , `custom.scss`).
- Los módulos propios: `ccproducteditor` , `ccfreesamplediscount` , `ccdesistimiento` , `ccproductreviews` , `addsampleoncatalog` , `customrelatedproducts` , `shippingcalculator` , `deliverypricecalculator` , `inspiration` , `inspirationcardsmodule` , `planatec` , `planatec_recomendaciones` , `outvio` , `ccdeliveryestimate` , `updatecatalog` y `manomanoorders` .
- Los scripts sueltos de la raíz del proyecto y de `utilsphp/` .

Cómo se ha hecho

- Lectura del código y búsquedas automáticas de patrones de riesgo: SQL construido con variables sin escapar, prefijo de tabla escrito a mano, endpoints sin token, `nofilter` en plantillas, llamadas PHP dentro de Smarty y estilos en línea.
- Pruebas contra la base de datos local en los puntos dudosos.
- Peticiones **de solo lectura** a producción para comprobar qué ficheros son accesibles desde fuera y qué cabeceras devuelve el servidor. No se ha ejecutado en producción ningún script que modifique datos.

Qué no cubre

- **No es una revisión visual completa.** Las sugerencias estéticas salen del código y de las capturas compartidas. Conviene validarlas en el navegador, y la tarea EST-05 propone medirlo con Lighthouse.
- **No se han revisado a fondo los módulos de terceros** (PayPal, Redsys, Seur, ets_megamenu, ybc_blog…). Se actualizan desde su proveedor.
- **No es una auditoría de seguridad con herramientas de ataque.** Los hallazgos de seguridad salen del código y de comprobaciones inofensivas.

Gravedad

Nivel	Significado
Crítica	Fuga de datos activa o riesgo inmediato
Alta	Fallo con impacto real en clientes o en los datos
Media	Fallo con impacto limitado, o riesgo que hoy no se da pero puede darse
Baja	Limpieza, mantenibilidad o mejora

1.2 Resumen

Lo urgente

- **Fuga de datos activa (crítica).** `https://ceramicconnection.com/modules/outvio/order.txt` es público y contiene el volcado completo de un pedido real. Además, ese fichero está subido al repositorio git.
- **Datos personales en un log de depuración (alta).** El paso de direcciones del checkout guarda en un fichero los datos de cada cliente que pasa por él.

Lo que más impacto tiene después

- **Lógica de negocio repartida y duplicada.**
  - Qué es una «muestra» se define de cuatro maneras distintas.
  - Los precios por m², cajas y descuentos se calculan en JavaScript.
  - Las validaciones de DNI y de código postal solo existen en JavaScript.
- **Riesgo de perder trabajo en el CSS.** Se ha editado el CSS compilado sin tocar el SCSS, y no hay un proceso de compilación reproducible.

Cifras del código

Indicador	Valor
`custom.js`	3.336 líneas (110 KB) dentro de un único `$(document).ready` · 29 `console.log`
`custom.scss`	9.361 líneas · 0 variables SCSS · 40 tamaños de letra distintos · 18 media queries distintas · 219 `!important`
`custom.css` compilado	258 KB, cargado en todas las páginas
Llamadas a PHP desde plantillas Smarty	100 líneas
Consultas SQL con el prefijo `ps_` escrito a mano	49 líneas
Carpetas `old` / `_old` / `_backup` / `_new` / `gitold`	78 carpetas, 190 ficheros en git

1.3 Seguridad y datos personales

S1 · Crítica · Volcado público de un pedido real

- `modules/outvio/outvio.php:904-905` escribe en cada pedido de Outvio `print_r($order)` y `print_r($carrier)` en `modules/outvio/order.txt` y `carrier.txt`. La línea 288 escribe `good.txt`.
- **En producción los tres responden 200.** Contienen, entre otros, `secure_key`, `id_cart`, `id_customer`, la referencia y los importes del último pedido (del 26/06/2026).
- Con el `id_cart` y la `secure_key` se puede abrir la página de confirmación de ese pedido.
- Los tres ficheros están además **subidos a git**, así que también están en el historial del repositorio.

S2 · Alta · Log con datos personales en el checkout

- `override/classes/checkout/CheckoutAddressesStep.php:9` llama a `debugAddressStepRequest()` (líneas 309-345) en **cada** petición del paso de direcciones.
- Esa función escribe la petición completa (nombre, dirección, teléfono, DNI…) en `var/logs/checkout_address_step_debug.log`.
- Es un tratamiento de datos personales sin finalidad ni plazo de borrado (RGPD), y el fichero crece sin límite.

S3 · Alta · SQL expuesto en una respuesta pública

- `modules/inspirationcardsmodule/controllers/front/filter.php:102` devuelve `'sql' => $sql` en el JSON que recibe cualquier visitante.
- Muestra la estructura de las tablas y facilita preparar ataques.

S4 · Alta · Scripts destructivos sin autenticación en el repositorio

- `utilsphp/bulkcombinationsimpleconverter.php` **borra combinaciones y stock** de unos 300 productos fijos con solo abrir la URL.
- `utilsphp/imagecloner.php` inserta imágenes.
- `utilsphp/info.php` y `phpinfo.php` muestran `phpinfo()`.
- `utilsphp/infouser.php` ejecuta `exec('whoami')`.
- `phppsinfo.php:8` tiene una contraseña escrita en el código.
- En producción `phpinfo.php` y `utilsphp/info.php` dan 404, así que **parece que no se suben**. Pero están en git, y un despliegue que copie el repositorio entero los publicaría. Los destructivos no se han probado en producción a propósito.

S5 · Media · Scripts de mantenimiento en la raíz pública

- Afecta a `fix_old_phones.php`, `listar-large-default.php`, `import_*.php` (cuatro scripts), `purge_sample_cart_rules.php` y `convert_custom_feature_values.php`.
- Todos se protegen con un token escrito en el código y subido a git.
  - `PFZ678CCE` se repite en dos scripts.
  - `3a257189…` se repite en cuatro.
- Quien tenga acceso al repositorio puede ejecutarlos desde fuera.

S6 · Media · Páginas de utilidad públicas sin control

- `pricelist.php`, `urlcategorylist.php`, `urlproductlist.php`, `urlimagelegendlist.php` y `viewproduct.php`.
- Cualquiera puede listar todo el catálogo, con descripciones, metadatos y URLs de imágenes. Hay que decidir si alguna es necesaria de verdad.

S7 · Media · Endpoint de cálculo de envío que escribe en la base de datos sin límite

- Ficheros: `modules/deliverypricecalculator/controllers/front/price.php` y `productestimate.php` (líneas 153-229).
- Cada petición crea una dirección y un carrito reales y los borra al terminar.
- No hay token ni límite de peticiones. Un bot puede generar miles de altas y bajas por minuto: carga en la base de datos y saltos en los autoincrementales.

S8 · Media · Faltan cabeceras de seguridad

- La portada responde con `X-Frame-Options` y `X-Content-Type-Options`.
- Faltan `Strict-Transport-Security`, `Content-Security-Policy`, `Referrer-Policy` y `Permissions-Policy`.

S9 · Media · Cookie de analítica antes del consentimiento

- La primera respuesta de la portada ya fija la cookie `ajs_anonymous_id` (Segment / analytics.js), antes de que el visitante acepte las cookies.
- La cookie `PrestaShop-…` se envía duplicada en la misma respuesta.

S10 · Baja · SQL construido sin conversión explícita

- Hoy los valores vienen de la base de datos o del propio código, no del visitante. Pero no hay defensa si eso cambia.
- Casos: `override/classes/Link.php:609` y `:621` (`LIKE "'.$alias.'"` sin `pSQL`), `override/classes/Cart.php:146` y `:159`, `override/classes/Group.php:45`,

`override/classes/Category.php:81` y `:99`, `override/classes/Product.php:445`, `:730` y `:769`.

- El más delicado es `modules/planatec/planatec.php:925`: inserta texto del formulario del admin sin escapar, así que una comilla rompe el guardado.

S11 · Baja · Controlador muerto

- `modules/addsampleoncatalog/controllers/front/ajax.php` siempre responde «Estoy Dentro».
- El código real está comentado. Tal como estaba escrito, dejaba añadir productos al carrito de cualquier cliente pasando su `id_cart`.

1.4 Fallos funcionales (bugs)

B1 · Alta · El umbral de DNI para clientes extranjeros falla en inglés

- `custom.js:2114-2125` (`getSubtotalProductsPriceRoundUpInteger`) lee el subtotal **del texto de la página** y lo interpreta con formato español: quita los puntos y cambia la coma por punto.
- En inglés (locale `en-US`) el importe se escribe «€2,345.00» y la función devuelve 3.
- Consecuencia: no se pide el DNI a un cliente extranjero que supera 2.200 €. El servidor sí lo comprueba (`Address::FOREIGN_ID_MIN_AMOUNT`, `CheckoutAddressesStep.php:56-63`) y bloquea más tarde con un error que el cliente no entiende.
- Además, el 2.200 está repetido a mano en cinco sitios del JavaScript.

B2 · Alta · Se ha editado el CSS compilado y no el SCSS

- `custom.css` tiene cinco commits del 05/10/2026 («sort and filters refactor», «fix carousel recommended products»…) sin cambios en `custom.scss`, cuyo último cambio es del 22/09.
- La próxima vez que alguien compile el SCSS, esos cambios **se perderán**.
- Tampoco existe un proceso de compilación: `make assets` llama a `tools/assets/build.sh`, que no existe, aunque `CLAUDE.md` lo documenta.

B3 · Media · Las reseñas solo se permiten si el pedido sigue en «Entregado»

- `modules/ccproductreviews/classes/Review.php:29-54` exige `o.current_state = entregado`. Si el pedido pasa después a otro estado, el cliente ya no puede opinar.
- Como ahora se factura desde Odoo, hay que comprobar si los estados se siguen actualizando en PrestaShop. Si no, **nadie puede dejar reseñas**.
- En `controllers/front/submit.php` queda el comentario «cambiar negación para poder hacer el submit», resto de una prueba.

B4 · Media · El peso se muestra con formato español en todos los idiomas

- `cart-detailed-totals.tpl` y `cart-summary-subtotals.tpl` formatean con `|string_format:"%.2f"|replace:'.':','`.
- En inglés sale «0,49 kg».

B5 · Media · Formatos de precio distintos en la misma pantalla

- En el carrito (captura del 13/10), la calculadora de envío muestra «8.01 €» con punto, y los totales «24,20 €» con coma.

B6 · Media · Aviso PHP en el Dispatcher

- `override/classes/Dispatcher.php:414-415` accede a `$req_uri[1]` sin comprobar que existe.
- Salta con peticiones sin ruta y con scripts de consola, y en modo desarrollo corta la ejecución (pasó al preparar el script de valores de características).

B7 · Media · Una regla de carrito por cada carrito con muestras

- `ccfreesamplediscount` crea una regla de carrito `CCFREESAMPLE-<id_cart>` para cada carrito.
- El propio `purge_sample_cart_rules.php` documenta 11,4 millones de filas en `ps_cart_rule_combination` y bloqueos al borrarlas.
- La purga es manual. El diseño de una regla por carrito es el origen del problema.

B8 · Baja · El envío no se oculta al volver a un paso anterior sin recargar

- El override de `Cart::getTotalShippingCost` ya muestra «Pendiente» fuera de los pasos Transporte y Pago.
- Pero el tema classic cambia de paso sin recargar al pulsar el título de un paso anterior. El resumen sigue mostrando el envío hasta la siguiente recarga.

B9 · Baja · Prefijo de tabla escrito a mano

- Hay 49 líneas con `ps_` en lugar de `_DB_PREFIX_`:

Fichero	Líneas
`override/classes/Product.php`	26
`modules/customrelatedproducts`	9
`override/classes/Category.php`	6
`override/classes/Cart.php`	5
`override/classes/tax/Tax.php`	2
`override/classes/order/OrderInvoice.php`	1

- Fallarían si se cambia el prefijo, por ejemplo en una migración o en un entorno de pruebas.

B10 · Baja · Módulos de pago solapados

- En la base de datos local están enganchados al pago a la vez `ps_checkout`, `paypal` y `paypalwithfee` (tres vías de PayPal), `redsys` (con pedidos también de `redsyspur`), `ps_checkpayment` (pago por cheque), `ps_wirepayment`, `revolutpayment` y `seurcashondelivery`.
- Hay que confirmarlo en producción. Si es así, hay duplicados que confunden al cliente y complican el mantenimiento.

1.5 Rendimiento

R1 · Media · `FrontController::assignGeneralPurposeVariables()` (override) hace trabajo innecesario en cada página

- Una consulta por cada valor de Formato y Color: unas 23 consultas en cada carga (`FrontController.php:142-150`).
- Carga la CMS 7 y la categoría destacada.
- Prepara los «productos sugeridos en el carrito» con el presentador completo de producto (`:357`, `getSuggestedProductsInCart`), aunque la página no los muestre.

R2 · Media · Consultas dentro de las plantillas

- `modules/ps_shoppingcart/modal.tpl` (líneas 54, 65, 159 y 336-338) lanza varias consultas **por producto** del carrito:
  - características;
  - categorías;
  - `isSample`;
  - nombre e imágenes de la junta recomendada.

- `ps_facetedsearch/.../facets.tpl:234` cuenta los productos con `Category::categoryProductsCountById()` **por cada subcategoría**.

R3 · Media · Comprobación de muestras con una consulta por línea

- `Product::isSample()` y `Cart::getSamplesNumberInCart()` lanzan una consulta por cada línea del carrito.
- Se llaman desde las plantillas del carrito en cada repintado.

R4 · Baja · Dos librerías de carrusel

- Owl Carousel (44 KB, depende de jQuery) y Swiper hacen lo mismo con aspecto y comportamiento distintos.

R5 · Baja · CSS completo en todas las páginas y `console.log` en producción

- `custom.css` (258 KB, sin minificar) se carga entero en todas las páginas.
- Quedan 29 `console.log`, algunos con datos del formulario de dirección (por ejemplo, código postal y provincia en `custom.js:2710-2711`).

1.6 Arquitectura y refactorización

A1 · Qué es una «muestra» se define de cuatro maneras distintas

Dónde	Criterio
`override/controllers/front/CartController.php:6`	Categoría 1751
`override/classes/Product.php:682-697` (`isSample`)	Característica 72 con valor 154747
`override/classes/Product.php:661-676` (`isSampleCombination`)	Combinación con `default_on` NULL
Pedidos	El nombre de la combinación («Muestra: Sí»)

- Cada parte del código usa una de ellas. Un cambio en el catálogo puede hacer que el carrito, el descuento y los informes dejen de coincidir.

A2 · Precios calculados en JavaScript

- `custom.js` calcula en el navegador:
  - el precio por cantidad con descuento (`getQuantityDiscount`, líneas 1294-1300 y 1368);
  - las cajas, piezas y m^2;
  - la calculadora de junta (líneas 1340-1365), con densidades de 850 y 750 kg/m^3 y el fabricante MAPEI (id 2) escritos a mano.
- El servidor tiene su propia lógica (precios específicos, overrides de `Cart` y `Product`). **Lo que ve el cliente en la ficha puede no coincidir con lo que cobra el carrito.** Tampoco se puede reutilizar en otros canales, como una futura app o API.

A3 · Validaciones solo en JavaScript

- `custom.js` comprueba en el navegador:
  - que el código postal corresponde a la provincia (`comprobarCPPorProvincia`, línea 2709, con un mapa de 51 códigos de provincia escrito a mano);
  - el formato de DNI, NIE y CIF (`validarDNI`, `validarNIE` e `isValidCif`, líneas 2781-2880).
- Si el JavaScript falla o se salta, el servidor acepta datos incorrectos que luego dan problemas en el envío y en la facturación.

A4 · `custom.js` es un único bloque de 3.336 líneas

- Todo vive dentro de un `$(document).ready` y se carga en todas las páginas.

- Mezcla carruseles, filtros, ficha de producto, carrito, formulario de dirección (unas 1.100 líneas, de la 2012 a la 3123) y checkout.

A5 · IDs escritos a mano y repetidos

- España = 6 y Francia = 8 (`CheckoutAddressesStep.php`, `custom.js:2323/2339/2366`).
- Característica 76 con valor 168467, de stock personalizado (`Cart.php`, `checkProductsOutOfStockInCart`).
- Un mapa de 33 valores de característica a categoría (`Link.php:203-247`).
- La categoría de muestras 1751.
- `CLAUDE.md` indica que estas constantes deberían vivir en `FrontController.php`.

A6 · Plantillas que llaman directamente a PHP

- Hay 100 líneas con llamadas `Clase::metodo()` dentro de los `.tpl`.
- Mezcla presentación y datos, dificulta la caché y es la causa de R2.

A7 · Estilos en línea

Plantilla	Atributos `style="…"`
`_modal-refund.tpl`	100
`product-add-to-cart.tpl`	80
`_modal-transport.tpl`	76
`product.tpl`	41

A8 · Código viejo dentro del proyecto

- Hay 78 carpetas `old`, `_old*`, `_backup*`, `_new` y `gitold`, con 190 ficheros en git. Por ejemplo, `assets/css/_backup_14_junio_2024` y `_backup_26_junio_2024`.
- Hay `.zip` dentro de `modules/` (`__paypalwithfee/paypalwithfee.zip`, `ccproductreviews.zip`, `orderfees_shipping.zip`).
- `XMLFeed.cache` y `error_log` están en git. `XMLFeed.cache` además es público.
- Git ya guarda el historial, así que estas copias solo confunden: aparecen en las búsquedas y se editan por error.

A9 · Traducciones fuera del sistema de PrestaShop

- `CheckoutAddressesStep.php:245` traduce con un array por `id_lang` (`1 => …, 2 => …`).
- `ccdesistimiento` tiene su propio sistema de traducciones (`ccL()` / `getCcTranslations()`, unas 400 líneas).
- Si cambian los IDs de idioma o se añade uno nuevo, los textos fallan sin avisar.

A10 · Herramientas de calidad

- `phpstan.neon.dist` solo analiza `src/` y `webservice/`, es decir, el núcleo de PrestaShop. No revisa nada del código propio (overrides, tema ni módulos).
- No hay pruebas automáticas del código propio.

1.7 Estética y experiencia de usuario

Estas sugerencias salen del código y de las capturas. Hay que validarlas viéndolas en el navegador.

- **E1 · El botón flotante de WhatsApp tapa contenido en el móvil.** En la captura del carrito cubre el importe de «Impuestos incluidos».
- **E2 · El bloque de entrega del carrito en el móvil es poco claro:**

    - «ENTREGA ESTIMADA: 07/09/2026 - 10/09/2026» ocupa todo el ancho, con una tipografía distinta a la de «Preparación» y «Envío»;
    - el peso total aparece antes que los artículos;
    - «Transporte: Pendiente» no explica cuándo se calculará.
- **E3 · No hay un sistema de diseño en el SCSS:** 0 variables, 18 colores en hexadecimal escritos a mano, 40 tamaños de letra distintos y 18 media queries con cortes diferentes. Es la causa de las pequeñas incoherencias visuales entre secciones.
- **E4 · Modales con estilos propios:** devolución, transporte y muestras se maquetan cada uno con estilos en línea, así que cada uno se ve un poco distinto.
- **E5 · No hay medición de referencia** de velocidad ni de accesibilidad (Lighthouse o Core Web Vitals) para saber si los cambios mejoran o empeoran.

Parte 2 · Tareas

Estimaciones: XS ≤ 1 h · S ≤ ½ día · M ≤ 2 días. Si una tarea M crece, conviene partirla.

Prioridades: Urgente (esta semana) · Alta · Media · Baja.

Formato de cada tarea:

- **Qué:** lo que hay que hacer.
- **Dónde:** ficheros y líneas.
- **Por qué:** el problema que resuelve.
- **Cómo:** los pasos.
- **Hecho cuando:** el criterio para cerrarla.

Las tareas están ordenadas por prioridad dentro de cada bloque. Las dependencias se indican con «Depende de».

Bloque SEC · Seguridad y datos personales

SEC-01 · Borrar los volcados públicos de pedidos de Outvio y dejar de generarlos

- **Prioridad:** Urgente · **Estimación:** XS · **Etiquetas:** seguridad, rgpd, outvio
- **Qué:** quitar las tres escrituras de depuración de Outvio y borrar los ficheros que ya existen, del servidor y de git.
- **Dónde:** `modules/outvio/outvio.php:288` (`good.txt`) y `:904-905` (`carrier.txt`, `order.txt`).
- **Por qué:** `https://ceramicconnection.com/modules/outvio/order.txt` es público (responde 200) y contiene el pedido completo de un cliente: `secure_key`, `id_cart`, referencia e importes. Se sobrescribe con cada pedido de Outvio.
- **Cómo:**
  1. Borrar las tres líneas `file_put_contents(...)`.
  2. Borrar `order.txt`, `carrier.txt` y `good.txt` en producción.
  3. Hacer `git rm` de esos ficheros.
  4. Decidir con el responsable de protección de datos si hay que notificar o valorar la exposición. El fichero existe al menos desde el 26/06/2026.
- **Hecho cuando:** las tres URLs responden 404, y un pedido nuevo con Outvio no vuelve a crear los ficheros.

SEC-02 · Bloquear desde Apache el acceso a ficheros de texto y logs dentro de `modules/`

- **Prioridad:** Urgente · **Estimación:** XS · **Etiquetas:** seguridad
- **Qué:** añadir una regla que deniegue `*.txt`, `*.log`, `*.sql`, `*.zip`, `*.bak` y `*.cache` bajo `modules/` y en la raíz.
- **Dónde:** `.htaccess` de la raíz, en un bloque propio fuera de la zona que regenera PrestaShop, o en la configuración de Cloudflare.
- **Por qué:** es una segunda barrera por si algún módulo vuelve a escribir un volcado. `XMLFeed.cache` también es público hoy.
- **Cómo:** `<FilesMatch "\.(txt|log|sql|zip|bak|cache)$"> Require all denied </FilesMatch>`, excepto `robots.txt` y los ficheros de verificación de dominio que se usen.
- **Hecho cuando:** `/modules/outvio/CHANGELOG.txt` y `/XMLFeed.cache` dan 403, y `/robots.txt` sigue dando 200.

SEC-03 · Quitar el log de depuración del paso de direcciones del checkout

- **Prioridad:** Urgente · **Estimación:** XS · **Etiquetas:** seguridad, rgpd, checkout
- **Qué:** eliminar la llamada a `debugAddressStepRequest()` y el propio método, y borrar el log que ya existe.
- **Dónde:** `override/classes/checkout/CheckoutAddressesStep.php:9` (llamada) y `:309-345` (método). Fichero: `var/logs/checkout_address_step_debug.log`.
- **Por qué:** guarda en cada petición los datos personales completos del cliente (nombre, dirección, DNI, teléfono), sin finalidad ni plazo de borrado (RGPD), y crece sin límite.
- **Cómo:** borrar la llamada y el método. Si hace falta depurar más adelante, registrar solo IDs y nunca valores del formulario. Borrar el `.log` en producción. Limpiar la caché de overrides.
- **Hecho cuando:** después de pasar por el checkout no se crea ni crece el fichero.

SEC-04 · No devolver la consulta SQL en el filtro de inspiraciones

- **Prioridad:** Urgente · **Estimación:** XS · **Etiquetas:** seguridad, inspirationcardsmodule
- **Qué:** quitar la clave `'sql' => $sql` de la respuesta JSON.
- **Dónde:** `modules/inspirationcardsmodule/controllers/front/filter.php:102`.
- **Por qué:** cualquier visitante ve la consulta interna y la estructura de las tablas.
- **Cómo:** borrar la línea. Comprobar en el JS del módulo que nada lee `response.sql`.
- **Hecho cuando:** la respuesta del filtro no contiene la clave `sql` y el filtro sigue funcionando.

SEC-05 · Eliminar del repositorio la carpeta `utilsphp/`

- **Prioridad:** Alta · **Estimación:** XS · **Etiquetas:** seguridad, limpieza
- **Qué:** `git rm -r utilsphp/`. Si algo se quiere conservar, pasarlo a la carpeta de scripts de SEC-07.
- **Dónde:**
  - `utilsphp/bulkcombinationsimpleconverter.php` (borra combinaciones y stock sin autenticación);
  - `imagecloner.php` y `__imagecloner.php` (insertan imágenes);
  - `info.php` (`phpinfo`);
  - `infouser.php` (`exec('whoami')`).
- **Por qué:** en producción dan 404, pero un despliegue que copie el repositorio entero los publicaría. Basta abrir la URL de `bulkcombinationsimpleconverter.php` para borrar datos de unos 300 productos.
- **Cómo:** confirmar con el equipo que nadie los usa. Borrarlos y comprobar que no existen en el servidor.
- **Hecho cuando:** la carpeta no existe ni en el repositorio ni en producción.

SEC-06 · Eliminar `phpinfo.php` y `phppsinfo.php`

- **Prioridad:** Alta · **Estimación:** XS · **Etiquetas:** seguridad, limpieza
- **Qué:** borrar los dos ficheros de la raíz.
- **Dónde:** `phpinfo.php`, y `phppsinfo.php:8`, que tiene una contraseña por defecto escrita en el código.
- **Por qué:** muestran la configuración del servidor. La contraseña está en git.
- **Cómo:** `git rm`. Si alguien necesita ver la configuración, usar Parámetros avanzados → Información del back-office.
- **Hecho cuando:** los dos ficheros no existen.

SEC-07 · Sacar los scripts de mantenimiento de la raíz pública

- **Prioridad:** Alta · **Estimación:** S · **Etiquetas:** seguridad, limpieza
- **Qué:** mover los scripts de un solo uso a una carpeta `scripts/` bloqueada desde la web, y hacer que solo se ejecuten por consola.
- **Dónde:** `fix_old_phones.php`, `listar-large-default.php`, `import_contenido_extra_categorias.php`, `import_horizontal_images*.php` (tres scripts), `purge_sample_cart_rules.php` y `convert_custom_feature_values.php`.
- **Por qué:** hoy se protegen con tokens escritos en el código y subidos a git (`PFZ678CCE` en dos scripts, `3a257189…` en cuatro). Cualquiera con acceso al repositorio puede ejecutarlos desde fuera.
- **Cómo:**
  1. Crear `scripts/` con un `.htaccess` `Require all denied`.
  2. Mover los scripts.
  3. Cambiar el arranque a `if (php_sapi_name() !== 'cli') exit;` y la ruta de `config.inc.php`.
  4. Quitar los tokens.
  5. Borrar los que ya se hayan ejecutado y no se vayan a repetir.
- **Hecho cuando:** la raíz solo tiene los ficheros propios de PrestaShop y los scripts funcionan con `php scripts/xxx.php`.

SEC-08 · Decidir qué hacer con las páginas de listado públicas de la raíz

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** seguridad, seo
- **Qué:** revisar para qué se usan `pricelist.php`, `urlcategorylist.php`, `urlproductlist.php`, `urlimagelegendlist.php` y `viewproduct.php`, y eliminarlas o protegerlas.
- **Dónde:** raíz del proyecto.
- **Por qué:** cualquiera puede listar el catálogo entero (descripciones, metadatos e imágenes), y hay herramientas de rastreo que las aprovechan.
- **Cómo:** preguntar a marketing o SEO si alguna se usa.
  - Las que se usen pasan a un controlador del back-office, por ejemplo una pestaña en `ccproducteditor`, o se protegen con la sesión del empleado.
  - El resto se borra.
- **Hecho cuando:** ninguna de esas URLs devuelve datos sin iniciar sesión.

SEC-09 · Poner token y límite de peticiones a la calculadora de envío

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** seguridad, rendimiento, deliverypricecalculator
- **Qué:** exigir el token estático de PrestaShop y limitar las peticiones por IP en los endpoints de cálculo.
- **Dónde:** `modules/deliverypricecalculator/controllers/front/price.php` y `productestimate.php`.
- **Por qué:** cada petición crea una dirección y un carrito reales en la base de datos (`productestimate.php:153-229`), sin ninguna protección. Un bot puede generar miles por minuto.
- **Cómo:**
  1. Enviar `static_token` desde el JS y comprobarlo con `Tools::getToken(false)`.
  2. Guardar un contador por IP en `Cache` o en una tabla pequeña (por ejemplo, 30 peticiones cada 10 minutos) y responder 429 al superarlo.
- **Hecho cuando:** sin token responde 403, y la petición 31 en 10 minutos desde la misma IP responde 429.

SEC-10 · Guardar en caché los cálculos de envío repetidos

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** rendimiento, deliverypricecalculator
- **Qué:** si llega el mismo cálculo (código postal, país, provincia y contenido del carrito o producto y cajas), devolver el resultado guardado en lugar de crear otro carrito temporal.
- **Dónde:** los mismos controladores que SEC-09.
- **Por qué:** reduce las escrituras en la base de datos y acelera la respuesta.
- **Cómo:** clave `md5(cp|país|provincia|checksum del carrito)`, guardada con `Cache::getInstance()->set()` durante 10 minutos.
- **Hecho cuando:** dos cálculos iguales seguidos solo crean un carrito temporal.
- **Depende de:** SEC-09.

SEC-11 · Comprobar que no quedan carritos ni direcciones temporales huérfanos

- **Prioridad:** Media · **Estimación:** XS · **Etiquetas:** datos, deliverypricecalculator
- **Qué:** consultar cuántos carritos y direcciones del cliente ficticio de la calculadora siguen en la base de datos, y borrarlos si los hay.
- **Dónde:** tablas `ps_cart` y `ps_address`, filtrando por el `id_customer` ficticio que usa `price.php`.
- **Por qué:** si PHP corta la ejecución (por ejemplo, por tiempo máximo), el borrado de `register_shutdown_function` puede no ejecutarse.
- **Cómo:** consulta de recuento. Si hay restos, un script de limpieza en `scripts/`.
- **Hecho cuando:** el recuento es 0, o se ha programado la limpieza.

SEC-12 · Añadir cabeceras de seguridad

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** seguridad, infraestructura
- **Qué:** activar `Strict-Transport-Security`, `Referrer-Policy: strict-origin-when-cross-origin` y `Permissions-Policy`, y `Content-Security-Policy` primero en modo `Report-Only`.
- **Dónde:** Cloudflare (Rules → Transform Rules → Response headers) o `.htaccess`.
- **Por qué:** hoy solo hay `X-Frame-Options` y `X-Content-Type-Options`. HSTS evita que alguien fuerce conexiones sin cifrar, y CSP limita el daño si alguna vez se cuela un script.
- **Cómo:**
  1. Poner HSTS con `max-age=31536000`, sin `preload` al principio.
  2. Poner la CSP en Report-Only durante dos semanas, revisar los avisos y añadir los dominios legítimos (PayPal, Redsys, Google, Cloudflare…) antes de aplicarla de verdad.
- **Hecho cuando:** las cabeceras aparecen en `curl -I https://ceramicconnection.com/es/`.

SEC-13 · Que la cookie `ajs_anonymous_id` no se cree antes del consentimiento

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** rgpd, cookies
- **Qué:** localizar qué módulo crea la cookie de Segment/analytics.js y hacer que espere al consentimiento.
- **Dónde:** se ve en la primera respuesta de la portada. Los candidatos son `ps_accounts`, `ps_checkout`, `psxmarketingwithgoogle` o algún módulo de marketing.
- **Por qué:** una cookie de analítica antes de aceptar las cookies incumple la normativa (LSSI/RGPD).
- **Cómo:** buscar `ajs_` en `modules/`. Desactivar la opción de telemetría del módulo, o condicionarlo al consentimiento del gestor de cookies.
- **Hecho cuando:** en una ventana privada, la primera visita no recibe `ajs_anonymous_id`.

SEC-14 · Convertir a entero o escapar los valores de las consultas SQL de los overrides

- **Prioridad:** Baja · **Estimación:** S · **Etiquetas:** seguridad, override
- **Qué:** añadir `(int)` o `pSQL()` a los valores que se concatenan en las consultas.
- **Dónde:** `override/classes/Link.php:609` y `:621` (`pSQL($alias)`), `Cart.php:146` y `:159`, `Group.php:45`, `Category.php:81` y `:99`, `Product.php:445`, `:730` y `:769`.
- **Por qué:** hoy los valores son internos, pero el código no se protege si un día llegan de la petición. Es la práctica estándar de PrestaShop.
- **Cómo:** cambio mecánico, línea a línea, y probar la ficha de producto, el carrito y el blog.
- **Hecho cuando:** ninguna consulta de `override/` concatena una variable sin `(int)`, `pSQL()` o `implode(',', array_map('intval', …))`.

SEC-15 · Escapar el texto que guarda el formulario de formatos de planatec

- **Prioridad:** Baja · **Estimación:** XS · **Etiquetas:** seguridad, planatec
- **Qué:** usar `pSQL()` o `Db::insert()` al guardar los valores del formulario.
- **Dónde:** `modules/planatec/planatec.php:925` (`VALUES (..., "' . $values[$name] . '")`).
- **Por qué:** un nombre con comillas rompe el guardado, y es una inyección SQL desde el back-office.
- **Cómo:** sustituir la consulta por `Db::getInstance()->insert('planatec_formatos', [...])`.
- **Hecho cuando:** se puede guardar un valor con comillas sin error.

SEC-16 · Eliminar el controlador muerto de `addsampleoncatalog`

- **Prioridad:** Baja · **Estimación:** XS · **Etiquetas:** limpieza, addsampleoncatalog
- **Qué:** borrar `controllers/front/ajax.php`, o terminarlo bien si alguien lo necesita.
- **Dónde:** `modules/addsampleoncatalog/controllers/front/ajax.php`.
- **Por qué:** siempre responde «Estoy Dentro». El código comentado aceptaba un `id_cart` de la petición y habría permitido modificar el carrito de otro cliente.
- **Cómo:** comprobar que nada llama a esa URL (buscar `addsampleoncatalog` y `ajax` en los JS) y borrarlo.
- **Hecho cuando:** el fichero no existe y añadir muestras desde el catálogo sigue funcionando.

Bloque BUG · Fallos funcionales

BUG-01 · Leer el subtotal del carrito como número, no del texto de la página

- **Prioridad:** Alta · **Estimación:** S · **Etiquetas:** bug, checkout, javascript, i18n
- **Qué:** pasar al JS el importe numérico de productos y dejar de leerlo del texto formateado.
- **Dónde:** `themes/child_classic/assets/js/custom.js:2114-2125` (`getSubtotalProductsPriceRoundUpInteger`). La plantilla del paso de direcciones o `cart-summary-subtotals.tpl`.
- **Por qué:** la función interpreta el texto con formato español. En inglés «€2,345.00» se convierte en 3, así que no se pide el DNI a extranjeros de más de 2.200 €. El servidor sí lo exige y bloquea después con un error.
- **Cómo:**
  1. Imprimir `data-products-amount="{$cart.subtotals.products.amount}"` (número sin formato) en el bloque del subtotal.
  2. Leerlo con `parseFloat(el.dataset.productsAmount)`.
  3. Borrar la lectura del texto.
- **Hecho cuando:** en inglés, con 2.300 € de productos y una dirección extranjera, el formulario pide el DNI.

BUG-02 · Tomar el umbral de 2.200 € del servidor y no repetirlo en el JS

- **Prioridad:** Alta · **Estimación:** XS · **Etiquetas:** bug, checkout, javascript
- **Qué:** exponer `Address::FOREIGN_ID_MIN_AMOUNT` y el ID de España al JS, y usar esas variables.
- **Dónde:** `override/classes/Address.php:7`. `custom.js:2323`, `:2339`, `:2366`, `:2902` y `:3088` (el 2200 y el `!= 6`).
- **Por qué:** si cambia el umbral en PHP, el JS se queda con el valor viejo, y el cliente ve un formulario que no coincide con lo que se le exige.
- **Cómo:** en `FrontController::setMedia()`, `Media::addJsDef(['ccCheckout' => ['foreignIdMinAmount' => Address::FOREIGN_ID_MIN_AMOUNT, 'spainCountryId' => …]])`. Sustituir los literales del JS.
- **Hecho cuando:** `custom.js` no contiene ni `2200` ni `!= 6`.
- **Depende de:** BUG-01.

BUG-03 · Pasar los cambios del CSS compilado al SCSS

- **Prioridad:** Alta · **Estimación:** M · **Etiquetas:** bug, css, deuda
- **Qué:** recuperar en `custom.scss` todo lo que se ha cambiado directamente en `custom.css` desde el 22/09.
- **Dónde:** `themes/child_classic/assets/css/custom.css`. Commits del 05/10/2026: 34ed90a2, 46498a39, 89940ba3, 04ec9fc8 y 9e4ace4d.
- **Por qué:** al recompilar el SCSS se perderán esos cambios: filtros, ordenación y carruseles de productos recomendados.
- **Cómo:**
  1. `git diff b51cc60c..HEAD -- themes/child_classic/assets/css/custom.css` para ver los cambios.
  2. Llevarlos a su sitio en `custom.scss`.
  3. Compilar y comparar el CSS resultante con el actual hasta que solo difieran en formato.
- **Hecho cuando:** compilar `custom.scss` da un `custom.css` sin diferencias visuales en la portada, una categoría, una ficha y el carrito.
- **Depende de:** REF-14 (proceso de compilación).

BUG-04 · Permitir reseñas aunque el pedido ya no esté en «Entregado»

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** bug, ccproductreviews
- **Qué:** comprobar que el pedido ha pasado alguna vez por el estado «Entregado», no que esté en él ahora.
- **Dónde:** `modules/ccproductreviews/classes/Review.php:29-54` (`customerCanReview`).
- **Por qué:** si el pedido pasa después a otro estado, el cliente ya no puede opinar. Hay que confirmar si, desde que se factura en Odoo, los estados siguen llegando a PrestaShop. Si no llegan, nadie puede dejar reseñas.
- **Cómo:** cambiar la condición por `EXISTS (SELECT 1 FROM ps_order_history oh WHERE oh.id_order = o.id_order AND oh.id_order_state = <entregado>)`. Contar cuántos pedidos han llegado a «Entregado» en los últimos 3 meses.
- **Hecho cuando:** un cliente con un pedido entregado y cambiado después de estado puede dejar su reseña.

BUG-05 · Quitar el comentario de prueba del envío de reseñas

- **Prioridad:** Baja · **Estimación:** XS · **Etiquetas:** limpieza, ccproductreviews
- **Qué:** borrar el comentario «cambiar negación para poder hacer el submit» y comprobar que la condición es la buena.
- **Dónde:** `modules/ccproductreviews/controllers/front/submit.php`, junto a `customerCanReview`.
- **Por qué:** es un resto de una prueba, e invita a alguien a «invertir» la comprobación.
- **Cómo:** borrarlo y probar que un cliente sin compra no puede enviar una reseña.
- **Hecho cuando:** no queda el comentario y la prueba pasa.

BUG-06 · Mostrar el peso del envío con el formato de cada idioma

- **Prioridad:** Media · **Estimación:** XS · **Etiquetas:** bug, i18n, carrito
- **Qué:** sustituir el formato español fijo por uno según el idioma.
- **Dónde:** `templates/checkout/_partials/cart-detailed-totals.tpl` y `cart-summary-subtotals.tpl` (`|string_format:"%.2f"|replace:'.':','`).
- **Por qué:** en inglés sale «0,49 kg».
- **Cómo:** calcular en PHP `Tools::getContextLocale($context)->formatNumber($peso)` y pasarlo ya formateado (ver REF-11). No llamar a `getTotalWeight()` desde la plantilla.
- **Hecho cuando:** en inglés sale «0.49 kg» y en español «0,49 kg».

BUG-07 · Formatear en el servidor el precio de la calculadora de envío

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** bug, i18n, shippingcalculator
- **Qué:** que la calculadora devuelva el precio ya formateado como el resto de precios.
- **Dónde:** la respuesta de `deliverypricecalculator/controllers/front/price.php` y el JS que pinta «8.01» junto a la caja «€».
- **Por qué:** en la misma pantalla del carrito salen «8.01 €» y «24,20 €» (captura del 13/10).
- **Cómo:** añadir a la respuesta un campo `price_formatted` generado con `Tools::getContextLocale()->formatPrice($importe, $isoMoneda)`, y pintarlo tal cual.
- **Hecho cuando:** la calculadora muestra «8,01 €» en español y «€8.01» en inglés.

BUG-08 · Evitar el aviso PHP del Dispatcher con rutas sin segmento

- **Prioridad:** Media · **Estimación:** XS · **Etiquetas:** bug, override
- **Qué:** comprobar que existe `$req_uri[1]` antes de usarlo.
- **Dónde:** `override/classes/Dispatcher.php:414-415`.
- **Por qué:** con peticiones sin ruta y con scripts de consola salta «Undefined offset: 1», y en modo desarrollo corta la ejecución.
- **Cómo:** `$segment = isset($req_uri[1]) ? $req_uri[1] : '';` y usar `$segment` en las líneas siguientes.
- **Hecho cuando:** `php -r "require 'config/config.inc.php'; require 'init.php';"` no da avisos con el modo desarrollo activo.

BUG-09 · Sustituir el prefijo `ps_` escrito a mano por `_DB_PREFIX_`

- **Prioridad:** Baja · **Estimación:** S · **Etiquetas:** deuda, override
- **Qué:** cambiar las 49 consultas con el nombre de tabla escrito a mano.
- **Dónde:** `override/classes/Product.php` (26), `modules/customrelatedproducts/customrelatedproducts.php` (9, por ejemplo líneas 639-663), `override/classes/Category.php` (6), `override/classes/Cart.php` (5, por ejemplo líneas 449 y 491), `override/classes/tax/Tax.php` (2) y `override/classes/order/OrderInvoice.php:114`.
- **Por qué:** fallan en cualquier entorno con otro prefijo (pruebas, migraciones).
- **Cómo:** cambio mecánico: `'ps_tabla'` → `'` . *DB_PREFIX* . `'tabla'`. Probar las páginas afectadas.
- **Hecho cuando:** la búsqueda de `` `ps_ `` en consultas de `override/` y de los módulos propios no da resultados.

BUG-10 · Revisar los módulos de pago activos y quitar los duplicados

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** pagos, configuración
- **Qué:** confirmar en producción qué módulos se muestran en el paso de pago y desactivar los que sobren.
- **Dónde:** Back-office → Pago. En local están activos `ps_checkout`, `paypal`, `paypalwithfee`, `redsys`, `ps_checkpayment` (cheque), `ps_wirepayment`, `revolutpayment` y `seurcashondelivery`.
- **Por qué:** tres vías distintas de PayPal y el pago por cheque confunden al cliente, y son más código que mantener y actualizar.
- **Cómo:** hacer una compra de prueba con un carrito de cada país y anotar qué opciones aparecen. Decidir con negocio cuáles se quedan y desinstalar el resto.
- **Hecho cuando:** solo queda una opción por cada método de pago (tarjeta, PayPal, transferencia, contra reembolso).

BUG-11 · Repintar el resumen del checkout al volver a un paso anterior

- **Prioridad:** Baja · **Estimación:** S · **Etiquetas:** checkout, javascript
- **Qué:** que al abrir un paso anterior pulsando su título se recargue el resumen del carrito, y el envío pase a «Pendiente».
- **Dónde:** el JS del checkout del tema (clic en `.checkout-step .step-title`). `override/classes/Cart.php` (`isShippingCostVisibleInCurrentContext`).
- **Por qué:** el tema classic cambia de paso sin recargar, así que el resumen sigue mostrando el envío hasta la siguiente recarga.
- **Cómo:** en ese clic, emitir `prestashop.emit('updateCart', {reason: {}})` o recargar la página con el parámetro del paso.
- **Hecho cuando:** al volver de Pago a Direcciones, el resumen muestra «Pendiente» sin tener que recargar.

Bloque RND · Rendimiento

RND-01 · Cargar las imágenes de formatos con una sola consulta y en caché

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** rendimiento, override
- **Qué:** sustituir el bucle de una consulta por formato por una sola consulta con `IN (...)`, y guardar el resultado en caché.
- **Dónde:** `override/classes/controller/FrontController.php:142-150` y `getFormatos()` (`:624-643`).
- **Por qué:** son unas 23 consultas en **cada** página de la tienda (12 formatos y 11 colores), y el resultado casi nunca cambia.
- **Cómo:** `SELECT id_formato, url FROM planatec_formatos WHERE id_formato IN (...)`. Guardarlo con `Cache::getInstance()->set('cc_img_formatos_'.$idLang, …, 3600)` y borrar esa clave al guardar formatos en `planatec`.
- **Hecho cuando:** una página de categoría hace como mucho una consulta a `planatec_formatos`.

RND-02 · Calcular los productos sugeridos solo en las páginas que los muestran

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** rendimiento, override
- **Qué:** llamar a `getSuggestedProductsInCart()` solo desde los controladores que lo usan.
- **Dónde:** `override/classes/controller/FrontController.php:357` y `:438`.
- **Por qué:** hoy se monta el presentador completo de varios productos en todas las páginas, incluida la portada y las CMS.
- **Cómo:** buscar en qué plantillas se usa `suggestedProductsInCart`. Pasar la asignación a esos controladores (`CartController`, el modal…).
- **Hecho cuando:** la variable solo se asigna en esas páginas y se ve igual que antes.

RND-03 · Sacar las consultas del modal de añadir al carrito

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** rendimiento, plantillas, ps_shoppingcart
- **Qué:** preparar en PHP los datos que el modal pide con llamadas desde la plantilla.
- **Dónde:** `themes/child_classic/modules/ps_shoppingcart/modal.tpl:54`, `:65`, `:159` y `:336-338`, `:361-377`.
- **Por qué:** por cada producto se consultan las características, las categorías, si es muestra, y el nombre, las imágenes y el enlace de la junta recomendada. Ralentiza el «añadir al carrito», que es el momento más sensible.
- **Cómo:** en el hook que pinta el modal (o con un override del controlador AJAX de `ps_shoppingcart`), calcular un array `modalData` con todo lo necesario y asignarlo a Smarty. La plantilla solo lo pinta.
- **Hecho cuando:** `modal.tpl` no contiene `Product::` ni `Context::`.

RND-04 · Sacar de la plantilla de filtros el recuento de productos por subcategoría

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** rendimiento, plantillas, ps_facetedsearch
- **Qué:** contar los productos de todas las subcategorías con una sola consulta en PHP.
- **Dónde:** `themes/child_classic/modules/ps_facetedsearch/views/templates/front/catalog/facets.tpl:30` y `:234`.
- **Por qué:** hoy se lanza una consulta por cada subcategoría al pintar los filtros de una categoría.
- **Cómo:** en un override o hook (por ejemplo `actionProductSearchAfter` o el `CategoryController` del override), calcular `[id_categoria => nº productos]` con `GROUP BY` y asignarlo a Smarty.
- **Hecho cuando:** `facets.tpl` no llama a `Category::`.

RND-05 · Unificar los carruseles en Swiper y retirar Owl Carousel

- **Prioridad:** Baja · **Estimación:** M · **Etiquetas:** rendimiento, javascript, estetica
- **Qué:** migrar los carruseles de Owl a Swiper y dejar de cargar `owl.carousel.min.js` y `.css`.
- **Dónde:** `themes/child_classic/assets/js/owl.carousel.min.js` y `assets/css/owl.carousel.min.css`. Plantillas que lo usan: `_partials/head.tpl`, `layouts/layout-both-columns.tpl`, `catalog/listing/product-list.tpl`, `catalog/_partials/product-cover-thumbnails.tpl`, `catalog/_partials/productlist_home.tpl` y `modules/ybc_blog/.../related_posts_category.tpl`. También los bloques de inicialización de `custom.js`.
- **Por qué:** dos librerías para lo mismo pesan más (44 KB solo Owl), Owl depende de jQuery, y cada carrusel se comporta distinto.
- **Cómo:** hacer una lista de los carruseles Owl (`grep owlCarousel`) y rehacerlos uno a uno con Swiper, con las mismas opciones.
- **Hecho cuando:** no se carga Owl en ninguna página y todos los carruseles funcionan con el dedo y el ratón.

RND-06 · Quitar los `console.log` de `custom.js`

- **Prioridad:** Baja · **Estimación:** XS · **Etiquetas:** javascript, rgpd, limpieza
- **Qué:** borrar los 29 `console.log`.
- **Dónde:** `themes/child_classic/assets/js/custom.js`. Por ejemplo, las líneas 2710-2711 sacan el código postal y la provincia.
- **Por qué:** ensucian la consola, muestran datos del formulario y dan pistas del código interno.
- **Cómo:** borrarlos, o envolverlos en `if (window.ccDebug)` si se quieren conservar para depurar.
- **Hecho cuando:** `grep -c console.log custom.js` da 0.

RND-07 · Rediseñar el descuento de muestras gratis para no crear una regla por carrito

- **Prioridad:** Media · **Estimación:** M · **Etiquetas:** rendimiento, datos, ccfreesamplediscount
- **Qué:** sustituir las reglas `CCFREESAMPLE-<id_cart>` por una regla compartida, o por un descuento calculado sin crear reglas.
- **Dónde:** `modules/ccfreesamplediscount/ccfreesamplediscount.php` (por ejemplo, `findModuleCartRuleForCart`, línea 571). `purge_sample_cart_rules.php`.
- **Por qué:** se crea una regla por cada carrito con muestras. El script de purga documenta 11,4 millones de filas en `ps_cart_rule_combination` y bloqueos al borrar. El problema vuelve a crecer cada día.
- **Cómo:** revisar qué condición hace única a cada regla. Si es el importe por carrito, valorar una sola regla con `reduction_product` o un hook de cálculo (`actionCartGetPackageShippingCost` u otro similar). Hasta entonces, programar la purga como tarea diaria (cron).
- **Hecho cuando:** el número de reglas `CCFREESAMPLE-%` deja de crecer con los carritos nuevos.

Bloque REF · Refactorización y orden del código

REF-01 · Crear una única definición de «muestra» en PHP

- **Prioridad:** Alta · **Estimación:** S · **Etiquetas:** refactor, muestras
- **Qué:** una clase `CcSample` (en `override/classes/` o en un módulo `cccore`) con `isSample($idProduct, $idProductAttribute = 0)` y `getSampleCategoryId()`. Debe documentar cuál es la regla oficial.
- **Dónde:** hoy está repartida en `CartController.php:6` (categoría 1751), `Product.php:682-697` (característica 72 con valor 154747) y `Product.php:661-676` (combinación con `default_on` NULL).
- **Por qué:** cuatro definiciones distintas hacen que el carrito, el descuento de muestras, los productos relacionados y los informes puedan discrepar.
- **Cómo:**
  1. Decidir con negocio cuál es la regla correcta.
  2. Implementarla en una sola clase, con consultas por lote: `areSamples(array $ids)`.
  3. Hacer que `Product::isSample` e `isSampleCombination` llamen a la nueva clase.
- **Hecho cuando:** existe la clase y los métodos antiguos la llaman.

REF-02 · Usar `CcSample` en el carrito, el descuento y los productos relacionados

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** refactor, muestras
- **Qué:** sustituir las comprobaciones de muestras por `CcSample`.
- **Dónde:** `override/controllers/front/CartController.php` (`sanitizeMuestrasCart`), `override/classes/Cart.php` (`getSamplesNumberInCart*`, que además hacen una consulta por línea), `modules/ccfreesamplediscount` y `modules/customrelatedproducts` (`$muestrasCategoryId`, línea 290).
- **Por qué:** es lo que da sentido a REF-01, y de paso elimina las consultas por línea (rendimiento R3).
- **Cómo:** cambiar las llamadas una a una y probar el carrito con muestras, el descuento de muestras gratis y los productos relacionados.
- **Hecho cuando:** `1751`, `154747` y `default_on` solo aparecen dentro de `CcSample`.
- **Depende de:** REF-01.

REF-03 · Reunir los IDs fijos (países, fabricantes, características) en una clase de configuración

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** refactor
- **Qué:** llevar los números mágicos a constantes con nombre, en `FrontController` (como indica `CLAUDE.md`) o en una clase `CcConfig`.
- **Dónde:**
  - España = 6 (`custom.js`);
  - Francia = 8 (`CheckoutAddressesStep.php`, `$FRANCE_ID`);
  - MAPEI = 2 (`custom.js`, calculadora de junta);
  - característica 76 con valor 168467 de stock personalizado (`Cart.php`, `checkProductsOutOfStockInCart`).
- **Por qué:** al leer el código no se sabe qué es cada número, y cuando cambia un ID hay que buscarlo a mano por todo el proyecto.
- **Cómo:** crear las constantes, sustituir los usos en PHP y exponer al JS con `Media::addJsDef` las que necesite.
- **Hecho cuando:** esos números solo aparecen en la definición de las constantes.

REF-04 · Pasar a la base de datos el mapa «valor de característica → categoría» del Link

- **Prioridad:** Baja · **Estimación:** S · **Etiquetas:** refactor, override
- **Qué:** sacar el array de 33 pares escrito en el código y guardarlo en configuración o en una tabla editable desde el back-office.
- **Dónde:** `override/classes/Link.php:203-247` (`getCategoryLinkByIdFeatureValue`) y `typeCategoryMap`.
- **Por qué:** cada vez que se crea un color o un acabado nuevo hay que tocar código y desplegar.
- **Cómo:** una tabla `cc_feature_value_category (id_feature_value, id_category)` con una pantalla sencilla, por ejemplo en `ccproducteditor`. `Link` la lee con caché.
- **Hecho cuando:** se puede añadir un par nuevo sin tocar código.

REF-05 · Documentar las reglas de precio que hoy calcula el JavaScript

- **Prioridad:** Alta · **Estimación:** S · **Etiquetas:** refactor, precios, documentacion
- **Qué:** escribir en un documento corto (`docs/reglas-precio.md`) cada regla de la ficha de producto:
  - precio por m^2 o por pieza;
  - redondeo a cajas completas;
  - metros lineales;
  - descuentos por cantidad;
  - muestras;
  - calculadora de junta.
- **Dónde:** `custom.js` (por ejemplo, las líneas 1290-1420) y `product-add-to-cart.tpl`.
- **Por qué:** es el paso previo para pasarlas a PHP sin cambiar ningún precio. Hoy esas reglas solo existen en el código.
- **Cómo:** recorrer cada calculadora del JS y anotar la fórmula, de dónde sale cada dato (característica 16, 17, 18, 82…) y un ejemplo con números reales.
- **Hecho cuando:** el documento existe y lo ha revisado alguien de negocio.

REF-06 · Crear el servicio PHP de cálculo de precio de la ficha, con pruebas

- **Prioridad:** Alta · **Estimación:** M · **Etiquetas:** refactor, precios
- **Qué:** una clase `CcProductPricing` que, dados un producto y una cantidad (m^2, piezas o cajas), devuelva cajas, m^2 reales, precio unitario, descuento aplicado y total. Usando los precios específicos de PrestaShop, no una copia propia.
- **Dónde:** nueva clase en `override/classes/` o en el módulo `cccore`.
- **Por qué:** hoy el precio que ve el cliente en la ficha lo calcula el navegador (`quantity * price * discount` , `custom.js:1294-1300`) y puede no coincidir con lo que cobra el carrito. Con una sola fuente, también se podría reutilizar en una futura API o app.
- **Cómo:** implementar las reglas de REF-05 con `Product::getPriceStatic()` y `SpecificPrice`. Escribir pruebas con 5 o 6 productos reales: por m^2, por pieza, con descuento por cantidad y lineal.
- **Hecho cuando:** las pruebas pasan, y para esos productos el resultado coincide con el que da hoy el carrito.
- **Depende de:** REF-05.

REF-07 · Que la ficha de producto use los datos del servicio de precio

- **Prioridad:** Alta · **Estimación:** M · **Etiquetas:** refactor, precios, javascript
- **Qué:** el JS de la ficha deja de calcular el precio. Pide el resultado al servidor, o usa una tabla calculada en PHP y entregada con la página.
- **Dónde:** `custom.js` (bloques de cantidad y cajas, `getQuantityDiscount`, piezas por caja), `product.tpl` y `product-add-to-cart.tpl`.
- **Por qué:** así lo que ve el cliente es exactamente lo que cobra el carrito.
- **Cómo:**
  - Opción A (sin peticiones): en `ProductController` (override), calcular con `CcProductPricing` los parámetros (precio base, tramos de descuento, m^2 por caja, piezas por caja) y entregarlos con `Media::addJsDef`. El JS solo multiplica con esos datos.
  - Opción B: un endpoint AJAX que devuelve el cálculo.
  - Empezar por la A.
- **Hecho cuando:** `custom.js` no tiene fórmulas de precio propias y, en los productos de prueba de REF-06, la ficha y el carrito coinciden.
- **Depende de:** REF-06.

REF-08 · Pasar la calculadora de junta a PHP

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** refactor, javascript
- **Qué:** llevar la fórmula de la junta (perímetro × ancho × espesor × densidad × 1,10) y las densidades a una clase PHP. El JS solo envía las medidas y pinta el resultado.
- **Dónde:** `custom.js:1340-1365` (densidades de 850 y 750 kg/m^3, MAPEI = 2 escrito a mano).
- **Por qué:** las densidades por fabricante son datos de negocio. Si se añade un fabricante nuevo, hay que tocar JavaScript.
- **Cómo:** guardar la densidad como característica del producto, o en un mapa por fabricante en `CcConfig`. Hacer un endpoint o incluir los parámetros en la página.
- **Hecho cuando:** la calculadora da los mismos sacos que antes para 3 casos de prueba, y `custom.js` no contiene densidades.

REF-09 · Validar en el servidor el DNI, el NIE y el CIF

- **Prioridad:** Alta · **Estimación:** S · **Etiquetas:** refactor, checkout, validacion
- **Qué:** una clase `CcIdValidator` con `isValidDni()`, `isValidNie()` e `isValidCif()`, usada al guardar la dirección.
- **Dónde:** hoy solo existe en `custom.js:2781-2880` (`validarDNI`, `validarNIE` e `isValidCif`). El override `Address.php` o el formulario de dirección.
- **Por qué:** si el JS falla o se salta, el servidor acepta documentos inválidos que luego dan problemas en las facturas de Odoo.
- **Cómo:** pasar a PHP los mismos algoritmos (letra de control) con pruebas. Engancharlos a la validación de `CustomerAddressForm` o a `Address::validateField`, y devolver el error en el idioma del cliente.
- **Hecho cuando:** enviar el formulario con JavaScript desactivado y un DNI incorrecto da error.

REF-10 · Validar en el servidor que el código postal corresponde a la provincia

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** refactor, checkout, validacion
- **Qué:** llevar a PHP la comprobación de los dos primeros dígitos del código postal según la provincia, y que el JS lea el mismo mapa.
- **Dónde:** `custom.js:2709` (`comprobarCPPorProvincia`, con 51 códigos de provincia escritos a mano por `id_state`).
- **Por qué:** sin validación en el servidor llegan direcciones incoherentes que dan errores de envío. El mapa está en el JS y ligado a `id_state`.
- **Cómo:** guardar el prefijo en `ps_state.iso_code` (si se puede) o en un mapa PHP. Validar al guardar la dirección y exponer el mapa al JS con `Media::addJsDef`.
- **Hecho cuando:** una dirección con CP 28001 y provincia Barcelona se rechaza también sin JavaScript.

REF-11 · Quitar las llamadas a PHP de las plantillas del carrito y del producto

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** refactor, plantillas
- **Qué:** preparar en el controlador los datos que hoy calculan las plantillas con `Clase::metodo()`.
- **Dónde:**
  - `cart-detailed-totals.tpl` y `cart-summary-subtotals.tpl` (`Context::getContext()->cart->getTotalWeight()`);
  - `cart-detailed-actions.tpl` (`Cart::getSamplesNumberInCartStatic`);
  - `catalog/product.tpl:99` (`Context::getContext()->cart->id`).
- **Por qué:** mezcla datos y presentación, dificulta la caché y lanza consultas en cada repintado.
- **Cómo:** asignar `cartWeight`, `samplesInCart` e `idCart` en `FrontController` (o solo en `CartController` y `OrderController`), y usar las variables en las plantillas.
- **Hecho cuando:** esas plantillas no contienen `Context::` ni `Cart::`.

REF-12 · Dividir `custom.js` en ficheros por página

- **Prioridad:** Media · **Estimación:** M · **Etiquetas:** refactor, javascript
- **Qué:** partir las 3.336 líneas en `common.js`, `listing.js`, `product.js`, `cart.js` y `checkout-address.js`, y cargar cada uno solo donde se usa.
- **Dónde:** `themes/child_classic/assets/js/custom.js`. `FrontController::setMedia()` (`:670-700`) para registrarlos según `$this->php_self`.
- **Por qué:** hoy se descarga y ejecuta todo en todas las páginas, y un error en un bloque puede romper el resto. Es difícil de mantener.
- **Cómo:**
  1. Mover los bloques sin cambiar su lógica (copiar y pegar).
  2. Registrar cada fichero con `registerJavascript` y la condición de página.
  3. Probar cada página.
  4. Hacerlo en varios commits, un fichero por commit.
- **Hecho cuando:** `custom.js` no existe o solo tiene lo común, y todas las páginas funcionan igual.
- **Depende de:** conviene hacerlo después de REF-07, REF-08 y REF-09, que eliminan código.

REF-13 · Pasar a clases SCSS los estilos en línea de los modales y del bloque de compra

- **Prioridad:** Baja · **Estimación:** S (por plantilla) · **Etiquetas:** refactor, css, estetica
- **Qué:** sustituir los atributos `style="…"` por clases en el SCSS. Una subtarea por plantilla.
- **Dónde:** `templates/catalog/_partials/_modal-refund.tpl` (100), `product-add-to-cart.tpl` (80), `_modal-transport.tpl` (76) y `catalog/product.tpl` (41).
- **Por qué:** los estilos en línea no se reutilizan, no responden a media queries y hacen que cada modal se vea distinto.
- **Cómo:** crear un componente `.cc-modal` común (cabecera, cuerpo, pie y botones) y clases para el bloque de compra.
- **Hecho cuando:** cada plantilla tiene menos de 5 atributos `style`, y se ve igual en el móvil y en el escritorio.

REF-14 · Crear un proceso de compilación del SCSS reproducible

- **Prioridad:** Alta · **Estimación:** S · **Etiquetas:** infraestructura, css
- **Qué:** un `package.json` en el tema con `sass` y un script `npm run build:css` (con el CSS minificado y el mapa de fuentes). `make assets` debe llamarlo.
- **Dónde:** `themes/child_classic/` y el `Makefile` de la raíz (que hoy llama a `tools/assets/build.sh`, que no existe). Corregir también `CLAUDE.md`.
- **Por qué:** hoy cada persona compila desde su editor, o directamente no compila (BUG-03), y el CSS que se sube (258 KB) no está minificado.
- **Cómo:** `npm i -D sass`. Script: `sass assets/css/custom.scss assets/css/custom.css --style=compressed --source-map`. Documentar en `CLAUDE.md` que **nunca** se edita `custom.css` a mano.
- **Hecho cuando:** `make assets` genera `custom.css` desde cero en cualquier equipo.

REF-15 · Dividir `custom.scss` en ficheros parciales

- **Prioridad:** Baja · **Estimación:** M · **Etiquetas:** refactor, css
- **Qué:** partir las 9.361 líneas en `_variables.scss`, `_base.scss`, `_header.scss`, `_listing.scss`, `_product.scss`, `_cart.scss`, `_checkout.scss`, `_blog.scss` y `_modals.scss`, con un `custom.scss` que solo los importa.
- **Dónde:** `themes/child_classic/assets/css/`.
- **Por qué:** con un solo fichero enorme se duplican reglas y es difícil encontrar dónde se define cada cosa. Hay 219 `!important` que nacen de reglas que se pisan entre sí.
- **Cómo:** mover los bloques sin cambiarlos, compilar y comparar el CSS resultante (debe ser idéntico).
- **Hecho cuando:** el CSS compilado es igual que antes y ningún parcial pasa de unas 1.500 líneas.
- **Depende de:** REF-14 y BUG-03.

REF-16 · Borrar del repositorio las carpetas de copias viejas del tema

- **Prioridad:** Baja · **Estimación:** XS · **Etiquetas:** limpieza
- **Qué:** `git rm -r` de las copias de seguridad del tema.
- **Dónde:** `themes/child_classic/assets/css/_backup_14_junio_2024`, `_backup_26_junio_2024`, `_new`, `old` y `.sass-cache`. `assets/js/_backup_14_junio_2024`, `_new` y `old`. Las `old/` de `templates/`.
- **Por qué:** git ya guarda el historial. Las copias confunden en las búsquedas y en el IDE, y se editan por error.
- **Cómo:** confirmar que nada las incluye (`grep -r "_backup\|/old/" templates`) y borrarlas.
- **Hecho cuando:** no quedan esas carpetas en el tema.

REF-17 · Borrar del repositorio las carpetas `old` de overrides y módulos, y los .zip

- **Prioridad:** Baja · **Estimación:** S · **Etiquetas:** limpieza
- **Qué:** quitar `override/**/old`, `override/**/_old_*`, `modules/*/old`, `gitold/` y los `.zip` de `modules/`.
- **Dónde:** 78 carpetas en total (190 ficheros en git). Los `.zip`: `modules/__paypalwithfee/paypalwithfee.zip`, `modules/ccproductreviews/ccproductreviews.zip` y `modules/orderfees_shipping/orderfees_shipping.zip`. Revisar también si `modules/__paypalwithfee` es una copia desactivada.
- **Por qué:** igual que REF-16. Además, PrestaShop puede llegar a cargar un override viejo por error si alguien lo renombra.
- **Cómo:** hacerlo en un commit aparte, solo de borrado, para poder revertirlo fácilmente.
- **Hecho cuando:** `find . -type d -name old -not -path ./vendor/*` no devuelve nada fuera de módulos de terceros.

REF-18 · Sacar de git los ficheros generados

- **Prioridad:** Baja · **Estimación:** XS · **Etiquetas:** limpieza
- **Qué:** `git rm --cached` de `XMLFeed.cache`, `error_log`, `custom.css.map` y `.sass-cache`, y añadirlos a `.gitignore`.
- **Dónde:** raíz y `themes/child_classic/assets/css/`.
- **Por qué:** cambian solos, generan conflictos y `XMLFeed.cache` es público.
- **Cómo:** añadirlos a `.gitignore` y `git rm --cached`.
- **Hecho cuando:** `git status` queda limpio después de generar el feed o compilar.

REF-19 · Usar el sistema de traducciones de PrestaShop en los textos del checkout

- **Prioridad:** Baja · **Estimación:** S · **Etiquetas:** refactor, i18n
- **Qué:** sustituir los arrays de textos por `id_lang` por `$this->trans()` o `$module->l()`.
- **Dónde:** `override/classes/checkout/CheckoutAddressesStep.php:242-250` (`getIncompleteAddressDniError`).
- **Por qué:** depende de que los IDs de idioma no cambien, y los textos no se pueden editar desde Internacional → Traducciones.
- **Cómo:** registrar las cadenas en el dominio `Shop.Notifications.Error` (o uno propio) y traducirlas desde el back-office.
- **Hecho cuando:** el texto se puede cambiar desde el back-office.

REF-20 · Pasar las traducciones de `ccdesistimiento` al sistema de PrestaShop

- **Prioridad:** Baja · **Estimación:** M · **Etiquetas:** refactor, i18n, ccdesistimiento
- **Qué:** sustituir `ccL()` y `getCcTranslations()` (unas 400 líneas de textos en el PHP) por `$this->l()` y los ficheros de traducción del módulo.
- **Dónde:** `modules/ccdesistimiento/ccdesistimiento.php:150-568`.
- **Por qué:** igual que REF-19. Son textos legales que el equipo no puede editar sin un desarrollador.
- **Cómo:** sustituir cada clave por `$this->l('texto en español')` y exportar e importar las traducciones de los demás idiomas.
- **Hecho cuando:** el módulo no tiene arrays de textos por idioma.

REF-21 · Ampliar PHPStan al código propio

- **Prioridad:** Baja · **Estimación:** S · **Etiquetas:** calidad, infraestructura
- **Qué:** añadir `override/` y los módulos propios a `phpstan.neon.dist`, empezando por el nivel 0.
- **Dónde:** `phpstan.neon.dist`, que hoy solo analiza `src/` y `webservice/`, es decir, el núcleo de PrestaShop.
- **Por qué:** detecta antes de subir errores como el aviso del Dispatcher (BUG-08) o variables sin definir.
- **Cómo:** añadir las rutas, generar el baseline (`--generate-baseline`) para no bloquear por lo existente, y subir de nivel poco a poco.
- **Hecho cuando:** `vendor/bin/phpstan analyse` pasa con las rutas nuevas.

Bloque EST · Estética y experiencia de usuario

EST-01 · Que el botón de WhatsApp no tape el carrito ni el checkout en el móvil

- **Prioridad:** Media · **Estimación:** XS · **Etiquetas:** estetica, movil
- **Qué:** separar el botón flotante del contenido en las páginas de carrito y pedido.
- **Dónde:** estilos del módulo `whatsappchat` (o su override en el SCSS) y el bloque de totales del carrito.
- **Por qué:** en el móvil tapa el importe de «Impuestos incluidos» (captura del carrito del 13/10), justo en la zona de los totales.
- **Cómo:** en `body#cart` y `body#checkout`, subir el botón por encima de la barra de acciones, o añadir `padding-bottom` al contenido con el alto del botón más `env(safe-area-inset-bottom)`.
- **Hecho cuando:** en un iPhone SE y en un Android mediano no tapa ningún importe ni botón.

EST-02 · Reordenar el bloque de totales del carrito en el móvil

- **Prioridad:** Media · **Estimación:** S · **Etiquetas:** estetica, carrito, movil
- **Qué:** poner primero artículos, descuentos, transporte y total, y después el peso (más discreto). Añadir a «Transporte: Pendiente» un texto de ayuda («Se calcula al elegir el transporte»).
- **Dónde:** `templates/checkout/_partials/cart-detailed-totals.tpl` y SCSS del carrito.
- **Por qué:** el peso aparece arriba, como si fuera lo más importante, y «Pendiente» sin explicación genera dudas y abandonos.
- **Cómo:** mover el bloque del peso debajo y en texto más pequeño. Añadir una línea de ayuda traducible bajo «Pendiente».
- **Hecho cuando:** se valida con una captura en el móvil y en el escritorio.

EST-03 · Unificar el estilo del bloque de plazo de entrega

- **Prioridad:** Baja · **Estimación:** XS · **Etiquetas:** estetica, movil
- **Qué:** que «ENTREGA ESTIMADA: fecha – fecha» use el mismo formato que «Preparación» y «Envío»: etiqueta pequeña y valor en negrita, en dos líneas en el móvil.
- **Dónde:** plantilla del módulo de plazo de entrega (`shippingcalculator` / `ccdeliveryestimate`) y su CSS.
- **Por qué:** hoy ocupa todo el ancho en mayúsculas, con otra tipografía, y en móviles estrechos se sale del borde.
- **Cómo:** aplicar la misma clase que las otras dos filas.
- **Hecho cuando:** las tres filas se ven iguales a 360 px de ancho.

EST-04 · Definir variables de diseño en el SCSS

- **Prioridad:** Baja · **Estimación:** S · **Etiquetas:** estetica, css
- **Qué:** crear `_variables.scss` con la paleta (los 18 colores actuales), una escala de 6 u 8 tamaños de letra, espaciados y 3 o 4 puntos de corte de media queries, y sustituir los valores escritos a mano.
- **Dónde:** `themes/child_classic/assets/css/custom.scss`: 0 variables, 40 tamaños de letra distintos y 18 media queries diferentes.
- **Por qué:** es la causa de las pequeñas diferencias de tamaños y márgenes entre secciones. Con variables, un cambio de marca se hace en un sitio.
- **Cómo:** sacar la lista de valores (`grep font-size`, colores en hexadecimal y `@media`), agrupar los parecidos (por ejemplo, 13 y 14 px pasan a uno solo), sustituirlos y revisarlo visualmente.
- **Hecho cuando:** no quedan colores en hexadecimal fuera de `_variables.scss`, y hay como mucho 8 tamaños de letra distintos.
- **Depende de:** REF-15.

EST-05 · Medir con Lighthouse antes y después de los cambios

- **Prioridad:** Media · **Estimación:** XS · **Etiquetas:** estetica, rendimiento, accesibilidad
- **Qué:** pasar Lighthouse en el móvil por la portada, una categoría, una ficha, el carrito y el paso de direcciones, y guardar los resultados como referencia.
- **Dónde:** Chrome DevTools → Lighthouse, o PageSpeed Insights. Guardar los informes en `docs/lighthouse/AAAA-MM-DD/`.
- **Por qué:** sin medir no se sabe si los cambios de RND y REF mejoran algo. Además saca errores de accesibilidad concretos: contraste, textos alternativos, tamaño de los botones.
- **Cómo:** ejecutarlo en modo incógnito y anotar el rendimiento, la accesibilidad, las buenas prácticas, el SEO, el LCP y el CLS de cada página. Crear una tarea por cada error de accesibilidad grave.
- **Hecho cuando:** existe la carpeta con los 5 informes y las tareas derivadas.

EST-06 · Dar el mismo diseño a todos los modales

- **Prioridad:** Baja · **Estimación:** S · **Etiquetas:** estetica, css
- **Qué:** que los modales de devolución, transporte y muestras compartan cabecera, márgenes, tipografía y botones.
- **Dónde:** `_modal-refund.tpl`, `_modal-transport.tpl` y `_modal-samples.tpl`.
- **Por qué:** cada uno está maquetado con sus propios estilos en línea, y se nota que son distintos.
- **Cómo:** aplicar el componente `.cc-modal` creado en REF-13.
- **Hecho cuando:** los tres modales se ven coherentes en una captura lado a lado.
- **Depende de:** REF-13.

Orden sugerido

1. **Esta semana:** SEC-01, SEC-02, SEC-03 y SEC-04. Son pequeñas y cierran fugas de datos activas.
2. **Siguiente iteración:** SEC-05, SEC-06 y SEC-07, BUG-01 y BUG-02, y REF-14 junto con BUG-03 antes de que alguien recompile el SCSS.
3. **Después:** el resto de SEC y BUG, y la serie de precios (REF-05 → REF-06 → REF-07), que es la refactorización de más valor.

4. **En paralelo, cuando haya huecos:** RND, la limpieza (REF-16 a REF-18) y EST.